

```
!pip install -U langgraph google-generativeai --quiet
```

```

_____ 43.7/43.7 kB 1.8 MB/s eta 0:00:00
_____ 154.8/154.8 kB 8.0 MB/s eta 0:00:00
_____ 43.9/43.9 kB 2.2 MB/s eta 0:00:00
_____ 56.8/56.8 kB 3.1 MB/s eta 0:00:00
_____ 216.7/216.7 kB 9.8 MB/s eta 0:00:00

```

```

import os, re, uuid, difflib
from getpass import getpass
from typing import TypedDict, List, Dict, Any
from langgraph.graph import StateGraph, START
import google.generativeai as genai

# Enter API key safely (or use Colab secrets)
api_key = getpass("Enter your GOOGLE_API_KEY (won't be echoed): ")
os.environ["GOOGLE_API_KEY"] = api_key
genai.configure(api_key=api_key)

# Lightweight wrapper for Gemini calls (safe fallback to empty string)
def gemini_generate(prompt: str, model_name="gemini-2.5-flash", max_output_tokens=512) -> str:
    try:
        model = genai.GenerativeModel(model_name)
        resp = model.generate_content(prompt) # adapt if your client requires kwargs
        return resp.text.strip() if resp and getattr(resp, "text", None) else ""
    except Exception as e:
        print("LLM error (gemini_generate):", e)
        return ""

```

Enter your GOOGLE_API_KEY (won't be echoed):

```

# Colab cell 3 – define the State (Ticket) type and a tiny KB for demo
class TicketState(TypedDict, total=False):
    ticket_id: str
    user_id: str
    subject: str
    description: str
    attachments: List[Any]
    priority: str
    category: str
    assigned_team: str
    kb_matches: List[Dict[str, Any]]
    resolution: str
    status: str
    suggested_reply: str
    escalate_reason: str
    logs: List[str]

# Example in-memory KB (in production use a vector DB / search engine)
SIMPLE_KB = [
    {"id": "kb_reset_pw", "title": "Reset password steps", "text": "To reset password: click forgot password, follow email link"},
    {"id": "kb_clear_cache", "title": "Clear browser cache", "text": "Instructions to clear cache on Chrome/Firefox..."},
    {"id": "kb_check_payment", "title": "Failed payment troubleshooting", "text": "Check card, retry charge, contact bank."},
]

```

```

# Colab cell 4 – node functions (explainable, small and testable)
def intake_node(state: TicketState) -> TicketState:
    ticket_id = state.get("ticket_id", str(uuid.uuid4()))
    logs = state.get("logs", []) + [f"intake:{ticket_id}"]
    return {
        "ticket_id": ticket_id,
        "user_id": state.get("user_id", "unknown"),
        "subject": state.get("subject", "").strip(),
        "description": state.get("description", "").strip(),
        "attachments": state.get("attachments", []),
        "priority": state.get("priority", ""), # may be set later
        "category": state.get("category", ""),
        "assigned_team": "",
        "kb_matches": [],
        "resolution": "",
        "status": "open",
        "suggested_reply": "",
        "escalate_reason": "",
        "logs": logs,
    }

def classify_priority_node(state: TicketState) -> Dict[str, Any]:
    # Fast rules-based priority classifier (safe, deterministic)
    text = (state.get("subject", "") + " " + state.get("description", "")).lower()

```

```

if any(k in text for k in ["outage", "down", "can't", "cannot", "unable to", "critical"]):
    priority = "urgent"
elif any(k in text for k in ["error", "failure", "bug", "not working", "issue"]):
    priority = "high"
else:
    priority = "normal"
logs = state.get("logs", []) + [f"priority_set:{priority}"]
return {"priority": priority, "logs": logs}

def triage_node(state: TicketState) -> Dict[str, Any]:
    # Basic mapping; if category missing, use LLM to infer category
    mapping = {"billing": "billing_team", "login": "auth_team", "performance": "infra_team", "feature": "product_team"}
    category = state.get("category", "").lower()
    if not category:
        # Optional: use LLM to infer category if none provided
        prompt = (
            "Classify the support ticket into one of: billing, login, performance, feature, general.\n"
            f"Ticket text: {state.get('subject', '')} {state.get('description', '')}\nCategory:"
        )
        llm_out = gemini_generate(prompt)
        cat = llm_out.splitlines()[0].strip().split()[0].lower() if llm_out else "general"
        category = cat if cat in mapping else "general"
    assigned = mapping.get(category, "support_team")
    logs = state.get("logs", []) + [f"triaged_to:{assigned}"]
    return {"category": category, "assigned_team": assigned, "logs": logs}

def kb_search_node(state: TicketState) -> Dict[str, Any]:
    # Simple fuzzy search across SIMPLE_KB (replace with embeddings/vector DB in production)
    query = (state.get("subject", "") + " " + state.get("description", "")).lower()
    best = (0.0, None)
    for a in SIMPLE_KB:
        score = difflib.SequenceMatcher(None, query, (a["text"] + " " + a["title"]).lower()).ratio()
        if score > best[0]:
            best = (score, a)
    matches = []
    if best[1] and best[0] > 0.35: # tunable threshold
        matches.append({"id": best[1]["id"], "title": best[1]["title"], "score": best[0], "text": best[1]["text"]})
    logs = state.get("logs", []) + [f"kb_matches:{len(matches)}"]
    return {"kb_matches": matches, "logs": logs}

def auto_resolve_node(state: TicketState) -> Dict[str, Any]:
    # Attempt auto resolve if KB match is strong and ticket not urgent
    matches = state.get("kb_matches", [])
    if matches and matches[0]["score"] > 0.75 and state.get("priority", "") != "urgent":
        resolution = f"Applied KB: {matches[0]['title']}"
        status = "resolved"
        suggested_reply = f"Hi - we've applied a suggested fix: {matches[0]['title']}. Please confirm if issue is resolved."
        logs = state.get("logs", []) + [f"auto_resolved:kb={matches[0]['id']}"]
        return {"resolution": resolution, "status": status, "suggested_reply": suggested_reply, "logs": logs}
    # otherwise no change
    return {}

def escalate_node(state: TicketState) -> Dict[str, Any]:
    # Simple escalate rule: if urgent and unresolved OR manual flag
    if state.get("status") != "resolved" and state.get("priority") == "urgent":
        assigned = "engineering_oncall"
        escalate_reason = "urgent_unresolved"
        logs = state.get("logs", []) + [f"escalated_to:{assigned}"]
        return {"assigned_team": assigned, "status": "escalated", "escalate_reason": escalate_reason, "logs": logs}
    return {}

def create_response_node(state: TicketState) -> Dict[str, Any]:
    # Use LLM to craft a nice human-friendly reply (or a template)
    if state.get("status") == "resolved":
        template = (
            "Write a short, polite message to the user confirming resolution.\n"
            f"Issue summary: {state.get('subject', '')} - {state.get('description', '')}\n"
            f"Resolution: {state.get('resolution', '')}\n\nReply:"
        )
        reply = gemini_generate(template)
        logs = state.get("logs", []) + ["reply_generated"]
        return {"suggested_reply": reply or state.get("suggested_reply", ""), "logs": logs}
    elif state.get("status") == "escalated":
        # Compose escalation note for human team
        note = (
            "ESCL: Ticket escalated. Provide context and steps tried:\n"
            f"Ticket: {state.get('ticket_id', '')}\nSubject: {state.get('subject', '')}\n"
            f"Desc: {state.get('description', '')}\nKB matches: {state.get('kb_matches', '')}\n"
        )
        logs = state.get("logs", []) + ["escalation_note_created"]
        return {"suggested_reply": note, "logs": logs}
    return {}

```

```
def close_ticket_node(state: TicketState) -> Dict[str, Any]:
    # If resolved -> closed; otherwise keep status
    if state.get("status") == "resolved":
        logs = state.get("logs", []) + ["closed"]
        return {"status": "closed", "logs": logs}
    return {}

def logging_node(state: TicketState) -> Dict[str, Any]:
    # Final audit log (placeholder - hook to DB)
    logs = state.get("logs", []) + [f"final_state:{state.get('status')}"]
    # In production: write state to DB here
    return {"logs": logs}
```

```
# Colab cell 5 - build + wire the graph
builder = StateGraph(TicketState)

# Register nodes (the builder uses the function name to identify nodes)
builder.add_node(intake_node)
builder.add_node(classify_priority_node)
builder.add_node(triage_node)
builder.add_node(kb_search_node)
builder.add_node(auto_resolve_node)
builder.add_node(escalate_node)
builder.add_node(create_response_node)
builder.add_node(close_ticket_node)
builder.add_node(logging_node)

# Wire edges (use function.__name__ to avoid hard-coded strings)
builder.add_edge(START, intake_node.__name__)
builder.add_edge(intake_node.__name__, classify_priority_node.__name__)
builder.add_edge(classify_priority_node.__name__, triage_node.__name__)
builder.add_edge(triage_node.__name__, kb_search_node.__name__)
# KB node branches to auto-resolve and escalate (logic inside nodes will decide)
builder.add_edge(kb_search_node.__name__, auto_resolve_node.__name__)
builder.add_edge(kb_search_node.__name__, escalate_node.__name__)
# both resolution and escalation go to reply generation
builder.add_edge(auto_resolve_node.__name__, create_response_node.__name__)
builder.add_edge(escalate_node.__name__, create_response_node.__name__)
builder.add_edge(create_response_node.__name__, close_ticket_node.__name__)
builder.add_edge(close_ticket_node.__name__, logging_node.__name__)

# set entry point and compile
builder.set_entry_point(intake_node.__name__)
graph = builder.compile()
```

```
# Colab cell 6 - run example ticket through graph
example_ticket = {
    "user_id": "user_1001",
    "subject": "Can't login - site down",
    "description": "I try to login and get 500 error. This is blocking all users.",
    # "priority": "urgent", # optional to provide from UI
}

result = graph.invoke(example_ticket)

print("=== FINAL STATE ===")
print("ticket_id:", result.get("ticket_id"))
print("status:", result.get("status"))
print("priority:", result.get("priority"))
print("assigned_team:", result.get("assigned_team"))
print("kb_matches:", result.get("kb_matches"))
print("resolution:", result.get("resolution"))
print("suggested_reply:", result.get("suggested_reply"))
print("logs:", "\n - ".join(result.get("logs", [])))
```

```

=== FINAL STATE ===
ticket_id: 0170d07e-8ee9-4ab6-81df-c5a10ff207ed
status: escalated
priority: urgent
assigned_team: engineering_oncall
kb_matches: []
resolution:
suggested_reply: ESCL: Ticket escalated. Provide context and steps tried:
Ticket: 0170d07e-8ee9-4ab6-81df-c5a10ff207ed
Subject: Can't login – site down
Desc: I try to login and get 500 error. This is blocking all users.
KB matches: []

logs: intake:0170d07e-8ee9-4ab6-81df-c5a10ff207ed
- priority_set:urgent
- triaged_to:auth_team
- kb_matches:0
- escalated_to:engineering_oncall
- escalation_note_created
- final_state:escalated

```

```

# Optional visualization (works in Colab if graph.draw method implemented)
try:
    from IPython.display import Image, display
    display(Image(graph.get_graph().draw_mermaid_png()))
except Exception as e:
    print("Graph visualization not available:", e)

```



